

Programação Funcional

Tipos Básicos e Programas Simples em Haskell

Vladimir Oliveira Di Iorio
DPI - UFV

Números Inteiros

- Na linguagem Haskell, o tipo de dados `Int` representa os números inteiros.
- O valor máximo de `Int` é 2147483647.
- O tipo `Integer` também representa números inteiros, sem restrição de tamanho.
- A vantagem de usar `Int` é um possível ganho em eficiência (depende da implementação).

Números Inteiros

- Operações aritméticas: +, -, *, /, div, mod, abs.
- Comparação: >, <, >=, <=, ==, /=.
- Exemplos:

```
Main> 5 + 3
```

```
8
```

```
Main> 10 <= 20
```

```
True
```

```
Main> 10 / 3
```

```
3.33333
```

Números Inteiros

```
Main> div 22 5
4
Main> mod 12 5
2
Main> 12 `mod` 5
2
Main> (+) 5 3
8
```

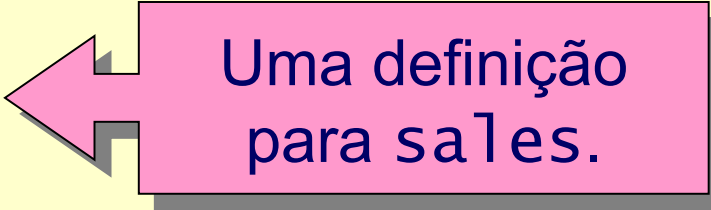
- Qualquer função com 2 parâmetros pode ser usada na forma infixa, usando ``.
- Operadores entre parêntesis são tratados como funções.

Programas com Inteiros

- Suponha uma função `sales :: Int -> Int`.
- A função `sales` retorna o número de unidades vendidas, de um determinado produto, se for fornecido o número do dia desejado.
- Os dias são numerados como 0,1,2 ...

```
1 sales :: Int -> Int
2 sales x
3   | x == 0      = 12
4   | x == 1      = 20
5   | x == 2      = 18
6   | x == 3      = 25
7   | otherwise   = 0
```

```
Main> sales 3
25
Main> sales 0
12
```



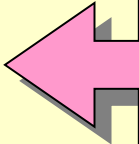
Uma definição
para `sales`.

Programas com Inteiros

- Suponha uma função `sales :: Int -> Int`.
- A função `sales` retorna o número de unidades vendidas, de um determinado produto, se for fornecido o número do dia desejado.
- Os dias são numerados como 0,1,2 ...

```
1 sales :: Int -> Int
2 sales x =
3   x `mod` 5
4   + (x+10) `mod` 7
```

```
Main> sales 3
9
Main> sales 0
3
```



Outra definição
para `sales`.

Programas com Inteiros

- Problema: definir $\text{totalSales} :: \text{Int} \rightarrow \text{Int}$, função que retorna o total de vendas até um determinado dia.

$$\text{totalSales } 2 = \text{sales } 0 + \text{sales } 1 + \text{sales } 2$$

$$\text{totalSales } n = \underline{\text{sales } 0 + \dots + \text{sales}(n-1)} + \text{sales } n$$

- Para $n=0$: $\text{totalSales } n = \text{sales } 0$

- Para $n>0$:

$$\text{totalSales } n = \underline{\text{totalSales } (n-1)} + \text{sales } n$$

Programas com Inteiros

```
8 totalSales :: Int -> Int
9 totalSales n
10   | n == 0      = sales 0
11   | otherwise   = totalSales (n-1) + (sales n)
```

```
1 sales :: Int -> Int
2 sales x
3   | x == 0      = 12
4   | x == 1      = 20
5   | x == 2      = 18
6   | x == 3      = 25
7   | otherwise   = 0
```

```
Main> totalSales 0
12
Main> totalSales 2
50
```

Programas com Inteiros

```
8  totalSales :: Int -> Int
9  totalSales n
10   | n == 0      = sales 0
11   | otherwise   = totalSales (n-1) + (sales n)
```

```
totalSales 2
= totalSales 1 + sales 2
= (totalSales 0 + sales 1) + sales 2
= (sales 0 + sales 1) + sales 2
```

Programas com Inteiros

```
1 sales :: Int -> Int
2 sales x
3   | x == 0      = 12
4   | x == 1      = 20
5   | x == 2      = 18
6   | x == 3      = 25
7   | otherwise   = 0
```

```
totalSales 2
= totalSales 1 + sales 2
= (totalSales 0 + sales 1) + sales 2
= (sales 0 + sales 1) + sales 2
= (12 + 20) + 18
= 50
```

Programas com Inteiros

- Problema: definir $\text{maxSales} :: \text{Int} \rightarrow \text{Int}$, função que retorna o valor máximo de unidades vendidas em um dia, em determinado período.

$\text{maxSales } n = \textit{máximo entre sales } 0, \dots, \text{sales } n$

- Para $n=0$: $\text{maxSales } n = \text{sales } 0$

- Para $n>0$:

$\text{maxSales } n = \textit{máximo entre maxSales}(n-1) \text{ e sales } n$

Programas com Inteiros

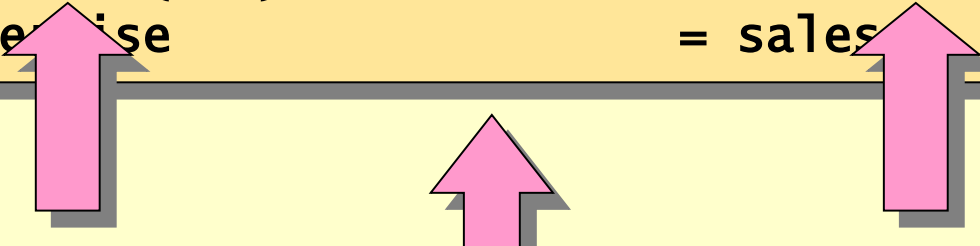
```
12 maxSales :: Int -> Int
13 maxSales n
14   | n == 0           = sales 0
15   | maxSales(n-1) >= sales n = maxSales(n-1)
16   | otherwise        = sales n
```

```
1 sales :: Int -> Int
2 sales x
3   | x == 0      = 12
4   | x == 1      = 20
5   | x == 2      = 18
6   | x == 3      = 25
7   | otherwise   = 0
```

```
maxSales 2
  ?? maxSales 1 >= sales 2
    ?? maxSales 0 >= sales 1
      ?? sales 0 >= sales 1
        ?? 12 >= 20
          = 20
        ?? 20 >= 18
          = 20
```

Programas com Inteiros

```
12 maxSales :: Int -> Int
13 maxSales n
14   | n == 0           = sales 0
15   | maxSales(n-1) >= sales n = maxSales(n-1)
16   | otherwise       = sales n
```



Ineficiência:
maxSales(n-1) é
calculada 2 vezes!

Programas com Inteiros

- Solução alternativa:

```
12 maxSales :: Int -> Int
13 maxSales n
14   | n == 0      = sales 0
15   | otherwise   = maxi (maxSales(n-1)) (sales n)
```

```
16 maxi :: Int -> Int -> Int
17 maxi m n
18   | m >= n      = m
19   | otherwise   = n
```

Programas com Inteiros

- Solução alternativa:

```
12 maxSales :: Int -> Int
13 maxSales n
14   | n == 0      = sales 0
15   | otherwise   = maxi (maxSales(n-1)) (sales n)
```

- Esta solução é mais clara que a primeira, destacando os casos da recursão ($n == 0$ e $n > 0$).
- É mais eficiente que a primeira, pois realiza menos cálculos.

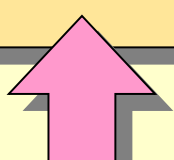
Reformulando Programas

```
8  totalSales :: Int -> Int
9  totalSales n
10   | n == 0      = sales 0
11   | otherwise   = totalSales (n-1) + (sales n)
```

```
totalSales -2
= totalSales (-3) + sales (-2)
= (totalSales (-4) + sales (-3)) + sales (-2)
= ...
```

Reformulando Programas

```
8 totalSales :: Int -> Int
9 totalSales n
10 | n == 0      = sales 0
11 | n > 0      = totalSales (n-1) + (sales n)
11 | otherwise  = 0
```



Definição mais
robusta para a função!

Reformulando Programas

```
8 totalSales :: Int -> Int
9 totalSales n
10   | n == 0      = sales 0
11   | n > 0       = totalSales (n-1) + (sales n)
11   | otherwise   = 0
```

- Ou também:

```
8 totalSales :: Int -> Int
9 totalSales n
10   | n >= 0      = totalSales (n-1) + (sales n)
11   | otherwise   = 0
```

Outros Programas

- Soma dos inteiros de 0 a n:

`sumi 0 = 0`

`sumi n = 0 + 1 + 2 + ... + (n-1) + n`

`sumi n = sumi (n-1) + n`

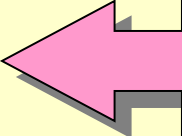
```
1 sumi :: Int -> Int
2 sumi n
3   | n <= 0      = 0
4   | otherwise   = n + sumi (n-1)
```

Outros Programas

- Fatorial de n:

```
1 fact :: Int -> Int
2 fact n
3   | n <= 0      = 1
4   | otherwise   = n * fact (n-1)
```

```
Main> fact 5
120
Main> fact 13
1932053504
Main> fact 14
1278945280
```



Extrapolou os
limites de Int.

Outros Programas

- Fatorial de n:

```
1 fact2 :: Integer -> Integer
2 fact2 n
3   | n <= 0      = 1
4   | otherwise   = n * fact2 (n-1)
```

```
Main> fact 13
1932053504
Main> fact2 13
6227020800
Main> fact2 30
2652528598121910586363084800000000
```

Sintaxe

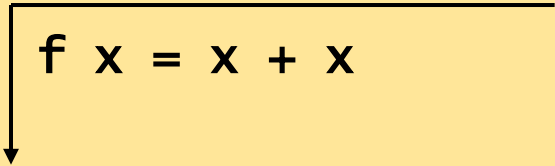
- Definições e layout:
 - Em Haskell, o layout de um programa é utilizado para determinar quando termina a definição de uma função e quando começa a próxima.
 - Uma definição termina com o primeiro elemento no mesmo nível de indentação (ou à esquerda deste).

Sintaxe

$f\ x = x + x$

Onde termina a definição
da função f ?

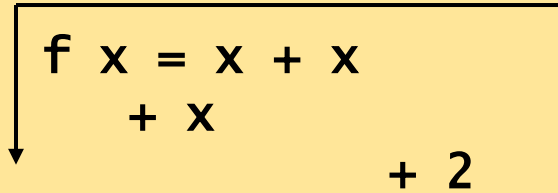
Sintaxe



$f\ x = x + x$

Até que apareça alguma definição nesse nível de indentação, todos os elementos pertencerão à definição de f .

Sintaxe



f x = x + x
+ x
+ 2

Sintaxe

f x = x + x
+ x
+ 2
g x y = ...

Sintaxe

```
f x = x + x
      + x
      + 2
g x y = ...
```

Sintaxe

- Definições e layout:
 - Outra forma para determinar o fim de uma definição é utilizar um símbolo terminador.
 - Em Haskell, o símbolo terminador é ";" .
 - Exemplo:

```
f x = x + x; g x y = x * y
```

Sintaxe

- Erro comum:

```
f x = x +  
1
```

- Mensagem de erro:

ERROR ... Syntax error in expression (unexpected ";", possibly due to bad layout)

Sintaxe

- Layout recomendado:

```
f v1 v2 ... vn
  | g1          = e1
  | g2          = e2
  ...
  | otherwise   = er      (ou | gr = er)
```

Sintaxe

- Quando expressões ou guardas são muito longas:

```
f v1 v2 ... vn
  | uma guarda muito longa que
    pode ocupar mais de uma linha
    = uma expressão muito longa que
      pode ocupar mais de uma linha
  | g2          = e2
  ...
```

Sintaxe

- Nomes em Haskell

- Começam com uma letra, seguida (opcionalmente) por letras, dígitos, "_" e "'";
- Começando com minúscula: funções e variáveis;
- Começando com maiúscula: nomes de módulos, nomes de tipos e construtores de tipo (ex: True e False).

- Palavras reservadas:

`case class data default deriving else hiding if
import in infix infixl infixr instance interface let
module of renaming then to type where`

Sintaxe

- Comentários de linha:
 - Símbolo usado: `--`
 - O texto à direita é considerado comentário.
- Comentários aninhados:
 - Símbolos usados: `{- -}`
 - Podem se estender por várias linhas.
 - Agem como parêntesis, podendo existir comentários dentro de outros comentários (aninhados).